```
================================================================================================================
FILE: backend/config.py
================================================================================================================
import os
from pathlib import Path
from typing import List
try:
    from dotenv import load_dotenv
except ImportError:
    load_dotenv = None
if load_dotenv:
    load_dotenv(Path(__file__).with_name(".env"))
def _string_list(name: str, default: str) -> List[str]:
    raw_value = os.getenv(name, default)
    return [item.strip() for item in raw_value.split(",") if item.strip()]
def _get_origins() -> List[str]:
    return _string_list(
        "CORS_ALLOW_ORIGINS",
        (
            "http://localhost:9080,http://127.0.0.1:9080,"
            "http://localhost:5173,http://127.0.0.1:5173"
        ),
    )
# GIS 交通路网教学知识图谱平台专属数据库配置。
NEO4J_URI = os.getenv("NEO4J_URI", "bolt://localhost:7687")
NEO4J_USER = os.getenv("NEO4J_USER", "neo4j")
NEO4J_PASSWORD = os.getenv("NEO4J_PASSWORD", "")
NEO4J_DATABASE = os.getenv("NEO4J_DATABASE", "traffic_kg")
# 教学知识图谱业务配置。
KG_CONFIG = {
    "max_learning_path_length": int(
        os.getenv("KG_MAX_LEARNING_PATH_LENGTH", "8")
    ),
    "enable_topology_check": os.getenv(
        "ENABLE_TOPOLOGY_CHECK",
        "true",
    ).strip().lower()
    in {"1", "true", "yes", "on"},
    "coordinate_tolerance": float(
        os.getenv("TOPOLOGY_COORDINATE_TOLERANCE", "1.0")
    ),
}
# GIS 交通路网教学问答引擎专属配置。
QA_CONFIG = {
    "max_references": int(os.getenv("ANSWER_MAX_REFERENCES", "6")),
    "rate_limit_per_minute": int(
        os.getenv("CHAT_RATE_LIMIT_PER_MINUTE", "5")
    ),
    "whitelist": _string_list("CHAT_RATE_LIMIT_WHITELIST", ""),
}
```

```
# 外部模型接口配置，只用于平台内部教学问答引擎。
DEEPSEEK_API_KEY = os.getenv("DEEPSEEK_API_KEY", "")
DEEPSEEK_BASE_URL = os.getenv("DEEPSEEK_BASE_URL", "https://api.deepseek.com")
DEEPSEEK_MODEL = os.getenv("DEEPSEEK_MODEL", "deepseek-chat")
DEEPSEEK_TIMEOUT_SECONDS = float(
    os.getenv("DEEPSEEK_TIMEOUT_SECONDS", "30")
)
DEEPSEEK_MAX_TOKENS = int(os.getenv("DEEPSEEK_MAX_TOKENS", "800"))
DEEPSEEK_TEMPERATURE = float(os.getenv("DEEPSEEK_TEMPERATURE", "0.2"))
CORS_ALLOW_ORIGINS = _get_origins()
================================================================================================
FILE: backend/main.py
================================================================================================
from contextlib import asynccontextmanager
import re
import threading
import time
from typing import Any, Dict, Iterator, List, Optional, Set, Tuple
from fastapi import FastAPI, HTTPException, Query, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from neo4j import GraphDatabase, READ_ACCESS
from neo4j.graph import Node, Relationship
from pydantic import BaseModel, Field
from pathlib import Path
import deepseek_service
import learning_path_engine
import topology_checker
from config import (
    CORS_ALLOW_ORIGINS,
    KG_CONFIG,
    NEO4J_DATABASE,
    NEO4J_PASSWORD,
    NEO4J_URI,
    NEO4J_USER,
    QA_CONFIG,
)
GRAPH_RELATIONSHIP_DEFINITIONS = {
    "BELONGS_TO": "知识点属于教学模块",
    "PREREQUISITE_OF": "前置知识点指向后续知识点",
    "RELATED_TO": "相关教学知识点",
    "PART_OF": "知识点组成关系",
    "APPLIED_IN": "知识应用于交通工程场景",
    "IMPLEMENTED_IN": "知识点通过工具或平台实现",
    "SUPPORTS": "模块对后续学习内容形成支撑",
    "HAS_MEDIA": "知识点关联视频或课件资源",
}
GRAPH_RELATIONSHIP_TYPES = list(GRAPH_RELATIONSHIP_DEFINITIONS)
MESSAGE_PREFIX = "GIS 交通路网教学知识图谱"
MEDIA_DIR = Path(__file__).resolve().parent / "media"
```

```
MEDIA_DIR.mkdir(parents=True, exist_ok=True)
CHAT_RATE_LOCK = threading.Lock()
CHAT_REQUEST_TIMES: Dict[str, List[float]] = {}
@asynccontextmanager
async def lifespan(app: FastAPI) -> Iterator[None]:
    app.state.driver = GraphDatabase.driver(
        NEO4J_URI,
        auth=(NEO4J_USER, NEO4J_PASSWORD),
    )
    try:
        yield
    finally:
        app.state.driver.close()
app = FastAPI(
    title="GIS 交通路网教学知识图谱平台",
    version="1.0",
    lifespan=lifespan,
)
app.add_middleware(
    CORSMiddleware,
    allow_origins=CORS_ALLOW_ORIGINS,
    allow_credentials=False,
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type"],
)
app.mount("/media", StaticFiles(directory=str(MEDIA_DIR)), name="media")
class TeachingQaRequest(BaseModel):
    question: str = Field(min_length=1, max_length=500)
    module_id: Optional[str] = Field(default=None, max_length=50)
    knowledge_point_id: Optional[str] = Field(default=None, max_length=50)
class TopologyCheckRequest(BaseModel):
    coordinate_tolerance: float = Field(
        default=KG_CONFIG["coordinate_tolerance"],
        ge=0,
        le=1000,
    )
def _get_driver(request: Request):
    return request.app.state.driver
def _business_id(node: Node) -> str:
    for key in ("module_id", "kp_id", "tool_id", "media_id"):
        value = node.get(key)
        if value is not None:
            return str(value)
    return node.element_id
def _property_value(value: Any) -> Any:
    if value is None or isinstance(value, (bool, int, float, str)):
        return value
    if isinstance(value, (list, tuple)):
        return [_property_value(item) for item in value]
    return str(value)
```

```
def _normalize_difficulty(value: Any) -> Optional[int]:
    return learning_path_engine.difficulty_rank(value) if value is not None else None
def _normalize_road_level(value: Any) -> Optional[str]:
    if value is None:
        return None
    level_map = {
        "0": "未分级",
        "1": "高速公路",
        "2": "城市快速路",
        "3": "城市主干路",
        "4": "城市次干路",
        "5": "城市支路",
        "motorway": "高速公路",
        "trunk": "城市快速路",
        "primary": "城市主干路",
        "secondary": "城市次干路",
        "local": "城市支路",
    }
    normalized = str(value).strip()
    return level_map.get(normalized.lower(), normalized)
def _validate_coordinate(properties: Dict[str, Any]) -> Optional[bool]:
    has_x = properties.get("x") is not None or properties.get("lon") is not None
    has_y = properties.get("y") is not None or properties.get("lat") is not None
    if not has_x and not has_y:
        return None
    try:
        x_value = properties.get("x", properties.get("lon"))
        y_value = properties.get("y", properties.get("lat"))
        x = float(x_value)
        y = float(y_value)
        return -180 <= x <= 180 and -90 <= y <= 90
    except (TypeError, ValueError):
        return False
def _serialize_node(node: Node) -> Dict[str, Any]:
    labels = sorted(node.labels)
    properties = {
        key: _property_value(value)
        for key, value in dict(node).items()
    }
    if "KnowledgePoint" in labels:
        properties["difficultyLevel"] = _normalize_difficulty(
            properties.get("difficulty")
        )
        coordinate_valid = _validate_coordinate(properties)
        if coordinate_valid is not None:
            properties["coordinateValid"] = coordinate_valid
    road_level = properties.get("road_level", properties.get("road_class"))
    if road_level is not None:
        properties["roadLevelLabel"] = _normalize_road_level(road_level)
    return {
```

```
        "id": _business_id(node),
        "elementId": node.element_id,
        "labels": labels,
        "properties": properties,
    }
def _serialize_relationship(relationship: Relationship) -> Dict[str, Any]:
    return {
        "id": relationship.element_id,
        "type": relationship.type,
        "source": _business_id(relationship.start_node),
        "target": _business_id(relationship.end_node),
        "properties": {
            key: _property_value(value)
            for key, value in dict(relationship).items()
        },
    }
class TrafficGraphBuilder:
    def __init__(self) -> None:
        self.nodes: List[Dict[str, Any]] = []
        self.links: List[Dict[str, Any]] = []
        self._node_ids: Set[str] = set()
        self._link_ids: Set[str] = set()
    def add_node(self, node: Optional[Node]) -> None:
        if node is None:
            return
        serialized = _serialize_node(node)
        if serialized["id"] in self._node_ids:
            return
        self._node_ids.add(serialized["id"])
        self.nodes.append(serialized)
    def add_relationship(self, relationship: Optional[Relationship]) -> None:
        if relationship is None:
            return
        serialized = _serialize_relationship(relationship)
        if serialized["id"] in self._link_ids:
            return
        self._link_ids.add(serialized["id"])
        self.links.append(serialized)
    def as_dict(self) -> Dict[str, Any]:
        return {
            "nodes": self.nodes,
            "links": self.links,
        }
def _read_query(
    driver,
    query: str,
    parameters: Optional[Dict[str, Any]] = None,
):
    resolved_parameters = parameters or {}
    with driver.session(
```

```
        database=NEO4J_DATABASE,
        default_access_mode=READ_ACCESS,
    ) as session:
        return list(session.run(query, resolved_parameters, timeout=8.0))
@app.get("/api/health")
def health(request: Request) -> Dict[str, Any]:
    try:
        _get_driver(request).verify_connectivity()
    except Exception as exc:
        raise HTTPException(
            status_code=503,
            detail="{0}Neo4j 连接失败: {1}".format(
                MESSAGE_PREFIX,
                str(exc),
            ),
        ) from exc
    return {
        "status": "ok",
        "message": "{0}后端服务正常".format(MESSAGE_PREFIX),
        "database": "connected",
    }
@app.get("/api/graph/modules")
def get_module_overview(request: Request) -> Dict[str, Any]:
    records = _read_query(
        _get_driver(request),
        """
        MATCH (module:Module)
        OPTIONAL MATCH (module)-[relationship]->(related:Module)
        RETURN module, relationship, related
        ORDER BY
          coalesce(module.sort_order, module.order, 9999),
          module.module_id
        """,
    )
    graph = TrafficGraphBuilder()
    for record in records:
        graph.add_node(record["module"])
        graph.add_node(record["related"])
        graph.add_relationship(record["relationship"])
    data = graph.as_dict()
    data["summary"] = {
        "moduleCount": len(
            [
                node
                for node in data["nodes"]
                if "Module" in node["labels"]
            ]
        ),
        "knowledgePointCount": None,
        "relationCount": len(data["links"]),
```

```
    }
    return {
        "code": 200,
        "message": "{0}模块图谱加载成功".format(MESSAGE_PREFIX),
        "data": data,
    }
@app.get("/api/directory")
def get_directory(request: Request) -> Dict[str, Any]:
    records = _read_query(
        _get_driver(request),
        """
        MATCH (module:Module)
        OPTIONAL MATCH (knowledge:KnowledgePoint)-[:BELONGS_TO]->(module)
        RETURN module, knowledge
        ORDER BY
          coalesce(module.sort_order, module.order, 9999),
          coalesce(knowledge.sort_order, 9999),
          knowledge.kp_id
        """,
    )
    modules_by_id: Dict[str, Dict[str, Any]] = {}
    module_order: List[str] = []
    for record in records:
        module = record["module"]
        module_id = _business_id(module)
        if module_id not in modules_by_id:
            module_data = _serialize_node(module)
            modules_by_id[module_id] = {
                "id": module_id,
                "name": module_data["properties"].get("name", "未命名模块"),
                "properties": module_data["properties"],
                "knowledgePoints": [],
            }
            module_order.append(module_id)
        knowledge = record["knowledge"]
        if knowledge is not None:
            knowledge_data = _serialize_node(knowledge)
            modules_by_id[module_id]["knowledgePoints"].append(
                {
                    "id": knowledge_data["id"],
                    "name": knowledge_data["properties"].get(
                        "name",
                        "未命名知识点",
                    ),
                    "properties": knowledge_data["properties"],
                }
            )
    modules = [modules_by_id[module_id] for module_id in module_order]
    return {
        "code": 200,
```

```
        "message": "{0}目录加载成功".format(MESSAGE_PREFIX),
        "data": {"modules": modules},
    }
@app.get("/api/modules/{module_id}/graph")
def get_module_graph(
    module_id: str,
    request: Request,
    internal_relation_limit: int = Query(default=800, ge=1, le=2000),
) -> Dict[str, Any]:
    driver = _get_driver(request)
    module_records = _read_query(
        driver,
        """
        MATCH (module:Module {module_id: $module_id})
        OPTIONAL MATCH (knowledge:KnowledgePoint)-[belongs:BELONGS_TO]->(module)
        RETURN module, knowledge, belongs
        ORDER BY coalesce(knowledge.sort_order, 9999), knowledge.kp_id
        """,
        {"module_id": module_id},
    )
    if not module_records:
        raise HTTPException(
            status_code=404,
            detail="{0}未找到指定模块".format(MESSAGE_PREFIX),
        )
    graph = TrafficGraphBuilder()
    knowledge_ids: List[str] = []
    for record in module_records:
        graph.add_node(record["module"])
        if record["knowledge"] is not None:
            graph.add_node(record["knowledge"])
            knowledge_ids.append(_business_id(record["knowledge"]))
        graph.add_relationship(record["belongs"])
    if knowledge_ids:
        relation_records = _read_query(
            driver,
            """
            MATCH (source:KnowledgePoint)-[relationship]->(target:KnowledgePoint)
            WHERE source.kp_id IN $knowledge_ids
              AND target.kp_id IN $knowledge_ids
              AND type(relationship) IN $relationship_types
            RETURN source, relationship, target
            LIMIT $limit
            """,
            {
                "knowledge_ids": knowledge_ids,
                "relationship_types": GRAPH_RELATIONSHIP_TYPES,
                "limit": internal_relation_limit,
            },
        )
```

```
        for record in relation_records:
            graph.add_node(record["source"])
            graph.add_node(record["target"])
            graph.add_relationship(record["relationship"])
    data = graph.as_dict()
    data["moduleId"] = module_id
    data["summary"] = {
        "nodeCount": len(data["nodes"]),
        "knowledgePointCount": len(knowledge_ids),
        "relationCount": len(data["links"]),
    }
    return {
        "code": 200,
        "message": "{0}模块知识点图谱加载成功".format(MESSAGE_PREFIX),
        "data": data,
    }
@app.get("/api/knowledge-points/{knowledge_point_id}")
def get_knowledge_point(
    knowledge_point_id: str,
    request: Request,
) -> Dict[str, Any]:
    driver = _get_driver(request)
    detail_records = _read_query(
        driver,
        """
        MATCH (knowledge:KnowledgePoint {kp_id: $knowledge_point_id})
        OPTIONAL MATCH (knowledge)-[:BELONGS_TO]->(module:Module)
        RETURN knowledge, module
        """,
        {"knowledge_point_id": knowledge_point_id},
    )
    if not detail_records:
        raise HTTPException(
            status_code=404,
            detail="{0}未找到指定知识点".format(MESSAGE_PREFIX),
        )
    knowledge = detail_records[0]["knowledge"]
    module = detail_records[0]["module"]
    relation_records = _read_query(
        driver,
        """
        MATCH (knowledge:KnowledgePoint {kp_id: $knowledge_point_id})
        MATCH (knowledge)-[relationship]-(related)
        WHERE related:KnowledgePoint
           OR related:Module
           OR related:Tool
        RETURN relationship, related
        LIMIT 200
        """,
        {"knowledge_point_id": knowledge_point_id},
```

```
)
media_records = _read_query(
    driver,
    """
    MATCH (knowledge:KnowledgePoint {kp_id: $knowledge_point_id})
    OPTIONAL MATCH (knowledge)-[media_relationship]->(media)
    WHERE type(media_relationship) = 'HAS_MEDIA'
    RETURN media
    """,
    {"knowledge_point_id": knowledge_point_id},
)
related_items = []
seen_relations: Set[Tuple[str, str]] = set()
for record in relation_records:
    relationship = record["relationship"]
    related = record["related"]
    key = (relationship.element_id, _business_id(related))
    if key in seen_relations:
        continue
    seen_relations.add(key)
    related_items.append(
        {
            "type": relationship.type,
            "direction": (
                "outgoing"
                if _business_id(relationship.start_node)
                == knowledge_point_id
                else "incoming"
            ),
            "node": _serialize_node(related),
        }
    )
materials = {
    "videos": [],
    "slides": [],
}
media_records = sorted(
    media_records,
    key=lambda record: (
        record["media"].get("order", 9999)
        if record["media"] is not None
        else 9999,
        record["media"].get("media_id", "")
        if record["media"] is not None
        else "",
    ),
)
for record in media_records:
    media = record["media"]
    if media is None:
```

```
            continue
        serialized = _serialize_node(media)
        properties = serialized["properties"]
        item = {
            "id": serialized["id"],
            "title": properties.get("title", "未命名资源"),
            "type": properties.get("type", "slides"),
            "format": properties.get("format", ""),
            "url": properties.get("url", ""),
            "coverUrl": properties.get("cover_url"),
            "order": properties.get("order", 9999),
        }
        if item["type"] == "video":
            materials["videos"].append(item)
        else:
            materials["slides"].append(item)
    return {
        "code": 200,
        "message": "{0}知识点详情加载成功".format(MESSAGE_PREFIX),
        "data": {
            "node": _serialize_node(knowledge),
            "module": _serialize_node(module) if module else None,
            "relations": related_items,
            "materials": materials,
        },
    }
@app.get("/api/search")
def search_nodes(
    request: Request,
    keyword: str = Query(min_length=1, max_length=100),
    limit: int = Query(default=20, ge=1, le=50),
) -> Dict[str, Any]:
    records = _read_query(
        _get_driver(request),
        """
        MATCH (node)
        WHERE (node:Module OR node:KnowledgePoint)
          AND (
            toLower(coalesce(node.name, '')) CONTAINS toLower($keyword)
            OR toLower(coalesce(node.description, '')) CONTAINS toLower($keyword)
          )
        RETURN node
        ORDER BY
          CASE WHEN node:Module THEN 0 ELSE 1 END,
          coalesce(node.sort_order, node.order, 9999),
          node.name
        LIMIT $limit
        """,
        {"keyword": keyword, "limit": limit},
    )
```

```
    results = []
    for record in records:
        node = record["node"]
        serialized = _serialize_node(node)
        results.append(
            {
                "id": serialized["id"],
                "name": serialized["properties"].get("name", "未命名"),
                "type": "Module" if "Module" in node.labels else "KnowledgePoint",
                "moduleId": serialized["properties"].get("module_id"),
                "description": serialized["properties"].get("description"),
            }
        )
    return {
        "code": 200,
        "message": "{0}搜索完成".format(MESSAGE_PREFIX),
        "data": {"results": results},
    }
def _context_node(node: Node, module_id: Optional[str] = None) -> Dict[str, Any]:
    serialized = _serialize_node(node)
    properties = serialized["properties"]
    if "Module" in node.labels:
        node_type = "Module"
    elif "KnowledgePoint" in node.labels:
        node_type = "KnowledgePoint"
    else:
        node_type = "Tool"
    return {
        "id": serialized["id"],
        "name": properties.get("name", "未命名"),
        "type": node_type,
        "description": properties.get("description"),
        "moduleId": module_id or properties.get("module_id"),
        "difficultyRank": properties.get("difficultyLevel", 3),
    }
GIS_TRAFFIC_DOMAIN_TERMS = [
    "交通路网",
    "路网",
    "高速公路",
    "城市快速路",
    "主干路",
    "次干路",
    "支路",
    "拓扑",
    "伪节点",
    "断头路",
    "高架",
    "地下",
    "轨迹",
    "浮动车",
```

```
    "GPS",
    "线性参考",
    "动态分段",
    "空间分析",
    "最短路径",
    "可达性",
    "道路匹配",
    "地理信息系统",
]
DOMAIN_SYNONYMS = {
    "高速": "高速公路",
    "快速路": "城市快速路",
    "GIS": "地理信息系统",
    "导航": "道路匹配",
    "路径规划": "最短路径",
    "轨迹数据": "轨迹",
}
def _extract_search_terms(question: str) -> List[str]:
    normalized = str(question or "").lower()
    stop_phrases = [
        "请介绍",
        "什么是",
        "有哪些",
        "是什么",
        "怎么",
        "如何",
        "推荐",
        "一条",
        "哪些",
        "以及",
        "关于",
        "学习路径",
        "请",
        "? ",
        "?",
        "。",
        "! ",
        "!",
    ]
    for phrase in stop_phrases:
        normalized = normalized.replace(phrase.lower(), " ")
    weighted_terms: Dict[str, Tuple[int, int]] = {}
    for domain_term in GIS_TRAFFIC_DOMAIN_TERMS:
        if domain_term.lower() in normalized:
            weighted_terms[domain_term] = (3, len(domain_term))
    for synonym, canonical in DOMAIN_SYNONYMS.items():
        if synonym.lower() in normalized:
            weighted_terms[canonical] = (2, len(canonical))
    parts = [
        part.strip()
```

```
        for part in re.split(r"[\s,, 、;; :: () () ]+", normalized)
        if len(part.strip()) >= 2
    ]
    for part in parts:
        if not any(part in term or term in part for term in weighted_terms):
            weighted_terms[part] = (1, len(part))
    ordered = sorted(
        weighted_terms.items(),
        key=lambda item: (
            -item[1][0],
            -item[1][1],
            item[0],
        ),
    )
    return [term for term, _ in ordered[:3]]
def _check_chat_rate_limit(request: Request) -> None:
    client_ip = request.headers.get(
        "x-real-ip",
        request.client.host if request.client else "unknown",
    )
    if "*" in QA_CONFIG["whitelist"] or client_ip in QA_CONFIG["whitelist"]:
        return
    now = time.time()
    window_start = now - 60
    with CHAT_RATE_LOCK:
        request_times = [
            timestamp
            for timestamp in CHAT_REQUEST_TIMES.get(client_ip, [])
            if timestamp >= window_start
        ]
        if len(request_times) >= QA_CONFIG["rate_limit_per_minute"]:
            raise HTTPException(
                status_code=429,
                detail=(
                    "{0}教学问答请求过于频繁，请稍后再试。".format(
                        MESSAGE_PREFIX
                    )
                ),
            )
        request_times.append(now)
        CHAT_REQUEST_TIMES[client_ip] = request_times
def _build_chat_context(
    driver,
    query: TeachingQaRequest,
) -> Dict[str, Any]:
    context: Dict[str, Any] = {
        "modules": [],
        "selectedModule": None,
        "selectedKnowledgePoint": None,
        "relatedKnowledgePoints": [],
```

```
    "searchMatches": [],
    "prerequisiteChain": [],
    "learningPath": [],
}
with driver.session(
    database=NEO4J_DATABASE,
    default_access_mode=READ_ACCESS,
) as session:
    module_records = list(
        session.run(
            """
            MATCH (module:Module)
            RETURN module
            ORDER BY
              coalesce(module.sort_order, module.order, 9999),
              module.module_id
            """,
            timeout=8.0,
        )
    )
    context["modules"] = [
        _context_node(record["module"])
        for record in module_records
    ]
    if query.module_id:
        module_record = session.run(
            """
            MATCH (module:Module {module_id: $module_id})
            RETURN module
            """,
            {"module_id": query.module_id},
            timeout=8.0,
        ).single()
        if module_record:
            context["selectedModule"] = _context_node(
                module_record["module"]
            )
    if query.knowledge_point_id:
        knowledge_record = session.run(
            """
            MATCH (knowledge:KnowledgePoint {kp_id: $kp_id})
            OPTIONAL MATCH (knowledge)-[:BELONGS_TO]->(module:Module)
            RETURN knowledge, module
            """,
            {"kp_id": query.knowledge_point_id},
            timeout=8.0,
        ).single()
        if knowledge_record:
            module = knowledge_record["module"]
            module_id = _business_id(module) if module else None
```

```
context["selectedKnowledgePoint"] = _context_node(
    knowledge_record["knowledge"],
    module_id,
)
if module and not context["selectedModule"]:
    context["selectedModule"] = _context_node(module)
relation_records = list(
    session.run(
        """
        MATCH (knowledge:KnowledgePoint {kp_id: $kp_id})
        MATCH (knowledge)-[relationship]-(related)
        WHERE related:KnowledgePoint
           OR related:Module
           OR related:Tool
        RETURN relationship, related
        LIMIT 24
        """,
        {"kp_id": query.knowledge_point_id},
        timeout=8.0,
    )
)
context["relatedKnowledgePoints"] = [
    {
        "relationship": record["relationship"].type,
        **_context_node(record["related"]),
    }
    for record in relation_records
]
context["relatedKnowledgePoints"].sort(
    key=lambda item: (
        item.get("properties", {}).get(
            "difficultyLevel",
            item.get("difficultyRank", 3),
        ),
        item.get("name", ""),
    )
)
learning_engine = learning_path_engine.load_learning_graph(
    driver,
    NEO4J_DATABASE,
    KG_CONFIG["max_learning_path_length"],
)
context["prerequisiteChain"] = (
    learning_engine.prerequisite_chain(
        query.knowledge_point_id,
        KG_CONFIG["max_learning_path_length"],
    )
)
context["learningPath"] = (
    learning_engine.build_learning_path(
```

```
                        query.knowledge_point_id
                    )
                )
        search_terms = _extract_search_terms(query.question)
        if search_terms:
            search_records = list(
                session.run(
                    """
                    MATCH (node)
                    WHERE (node:Module OR node:KnowledgePoint)
                      AND any(
                        term IN $terms
                        WHERE toLower(coalesce(node.name, ''))
                                CONTAINS toLower(term)
                           OR toLower(coalesce(node.description, ''))
                                CONTAINS toLower(term)
                      )
                    RETURN node
                    LIMIT 18
                    """,
                    {"terms": search_terms},
                    timeout=8.0,
                )
            )
            context["searchMatches"] = [
                _context_node(record["node"])
                for record in search_records
            ]
    return context
def _prerequisite_chain_query(
    driver,
    knowledge_point_id: str,
    max_depth: int,
) -> List[Dict[str, Any]]:
    engine = learning_path_engine.load_learning_graph(
        driver,
        NEO4J_DATABASE,
        max_depth,
    )
    return engine.prerequisite_chain(knowledge_point_id, max_depth)
def _module_difficulty_query(
    driver,
    module_id: str,
    difficulty: str,
) -> List[Dict[str, Any]]:
    records = _read_query(
        driver,
        """
        MATCH (knowledge:KnowledgePoint)-[:BELONGS_TO]->
              (module:Module {module_id: $module_id})
```

```
        WHERE knowledge.difficulty = $difficulty
        RETURN knowledge
        ORDER BY
          coalesce(knowledge.sort_order, 9999),
          knowledge.kp_id
        """,
        {
            "module_id": module_id,
            "difficulty": difficulty,
        },
    )
    return [
        _context_node(record["knowledge"], module_id)
        for record in records
    ]
@app.get("/api/knowledge-points/{knowledge_point_id}/learning-path")
def get_learning_path(
    knowledge_point_id: str,
    request: Request,
    max_depth: int = Query(default=8, ge=1, le=20),
) -> Dict[str, Any]:
    path = _prerequisite_chain_query(
        _get_driver(request),
        knowledge_point_id,
        max_depth,
    )
    return {
        "code": 200,
        "message": "{0}知识点前置学习链加载成功".format(MESSAGE_PREFIX),
        "data": {
            "knowledgePointId": knowledge_point_id,
            "path": path,
        },
    }
@app.get("/api/modules/{module_id}/knowledge-points")
def get_module_knowledge_points(
    module_id: str,
    request: Request,
    difficulty: str = Query(min_length=1, max_length=20),
) -> Dict[str, Any]:
    items = _module_difficulty_query(
        _get_driver(request),
        module_id,
        difficulty,
    )
    return {
        "code": 200,
        "message": "{0}模块难度知识点筛选成功".format(MESSAGE_PREFIX),
        "data": {
            "moduleId": module_id,
```

```
            "difficulty": difficulty,
            "items": items,
        },
    }
@app.post("/api/topology/check")
def run_topology_check(
    request: Request,
    payload: Optional[TopologyCheckRequest] = None,
) -> Dict[str, Any]:
    if not KG_CONFIG["enable_topology_check"]:
        raise HTTPException(
            status_code=404,
            detail="{0}拓扑校验功能未启用。".format(MESSAGE_PREFIX),
        )
    records = _read_query(
        _get_driver(request),
        """
        MATCH (node)
        OPTIONAL MATCH (node)-[relationship]->(related)
        RETURN node, relationship, related
        LIMIT 2000
        """,
    )
    graph = TrafficGraphBuilder()
    for record in records:
        graph.add_node(record["node"])
        graph.add_node(record["related"])
        graph.add_relationship(record["relationship"])
    result = topology_checker.check_topology(
        graph.nodes,
        graph.links,
        (
            payload.coordinate_tolerance
            if payload is not None
            else KG_CONFIG["coordinate_tolerance"]
        ),
    )
    return {
        "code": 200,
        "message": "{0}拓扑完整性校验完成".format(MESSAGE_PREFIX),
        "data": result,
    }
@app.get("/api/chat/status")
def chat_status() -> Dict[str, Any]:
    return {
        "code": 200,
        "message": "{0}教学问答状态读取成功".format(MESSAGE_PREFIX),
        "data": deepseek_service.get_status(),
    }
@app.post("/api/chat")
```

```
def chat(
    query: TeachingQaRequest,
    request: Request,
) -> Dict[str, Any]:
    status = deepseek_service.get_status()
    if not status["configured"]:
        raise HTTPException(
            status_code=503,
            detail=(
                "{0}教学问答引擎尚未配置，请检查 backend/.env。".format(
                    MESSAGE_PREFIX
                )
            ),
        )
    _check_chat_rate_limit(request)
    graph_context = _build_chat_context(_get_driver(request), query)
    try:
        qa_result = deepseek_service.teaching_qa(
            query.question,
            graph_context,
        )
    except deepseek_service.QaEngineConfigError as exc:
        raise HTTPException(status_code=503, detail=str(exc)) from exc
    except deepseek_service.QaEngineRequestError as exc:
        raise HTTPException(status_code=502, detail=str(exc)) from exc
    return {
        "code": 200,
        "message": "{0}教学问答成功".format(MESSAGE_PREFIX),
        "answer": qa_result["answer"],
        "scene": qa_result.get("scene"),
        "references": qa_result.get("references", [])[
            : QA_CONFIG["max_references"]
        ],
        "model": status["model"],
    }
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(
        "main:app",
        host="0.0.0.0",
        port=18081,
        reload=True,
    )
================================================================================================
FILE: backend/topology_checker.py
================================================================================================
from collections import defaultdict
from math import hypot
from typing import Any, Dict, Iterable, List, Optional, Tuple
TOPOLOGY_RELATIONSHIPS = {
```

```
    "APPLIED_IN",
    "IMPLEMENTED_IN",
    "PART_OF",
    "PREREQUISITE_MODULE",
    "PREREQUISITE_OF",
    "RELATED_TO",
    "SUPPORTS",
}
LAYER_CONNECTOR_TYPES = {
    "INTERCHANGE",
    "RAMP",
    "LAYER_CONNECTOR",
}
def _node_id(node: Dict[str, Any]) -> str:
    if node.get("id") is not None:
        return str(node["id"])
    properties = node.get("properties", {})
    for key in ("module_id", "kp_id", "tool_id", "media_id", "road_id"):
        if properties.get(key) is not None:
            return str(properties[key])
    return ""
def _labels(node: Dict[str, Any]) -> List[str]:
    return [str(label) for label in node.get("labels", [])]
def _properties(node: Dict[str, Any]) -> Dict[str, Any]:
    properties = node.get("properties", node)
    return properties if isinstance(properties, dict) else {}
def _numeric(value: Any) -> Optional[float]:
    try:
        return float(value)
    except (TypeError, ValueError):
        return None
def _coordinate_from(properties: Dict[str, Any]) -> Optional[Tuple[float, float]]:
    x = _numeric(
        properties.get("x")
        if properties.get("x") is not None
        else properties.get("lon")
    )
    y = _numeric(
        properties.get("y")
        if properties.get("y") is not None
        else properties.get("lat")
    )
    if x is None or y is None:
        return None
    return x, y
def _layer_from(properties: Dict[str, Any]) -> Optional[str]:
    explicit_layer = str(properties.get("layer", "")).strip().lower()
    if explicit_layer:
        aliases = {
            "ground": "ground",
```

```
            "surface": "ground",
            "地面": "ground",
            "elevated": "elevated",
            "viaduct": "elevated",
            "高架": "elevated",
            "underground": "underground",
            "tunnel": "underground",
            "地下": "underground",
        }
        return aliases.get(explicit_layer, explicit_layer)
    elevation = _numeric(
        properties.get("elevation")
        if properties.get("elevation") is not None
        else properties.get("height")
    )
    if elevation is None:
        return None
    if elevation > 3:
        return "elevated"
    if elevation < -3:
        return "underground"
    return "ground"
def _is_intersection(properties: Dict[str, Any]) -> bool:
    value = properties.get("is_intersection", False)
    return value is True or str(value).strip().lower() in {"1", "true", "yes"}
def _issue(
    issue_type: str,
    severity: str,
    message: str,
    node_ids: Optional[Iterable[str]] = None,
    relationship_id: Optional[str] = None,
    suggestion: str = "",
) -> Dict[str, Any]:
    return {
        "type": issue_type,
        "severity": severity,
        "message": message,
        "nodeIds": list(node_ids or []),
        "relationshipId": relationship_id,
        "suggestion": suggestion,
    }
def identify_pseudo_nodes(
    nodes: List[Dict[str, Any]],
    relationships: List[Dict[str, Any]],
) -> List[Dict[str, Any]]:
    degrees: Dict[str, int] = defaultdict(int)
    node_lookup = {_node_id(node): node for node in nodes}
    for relationship in relationships:
        if relationship.get("type") not in TOPOLOGY_RELATIONSHIPS:
            continue
```

```
        source = str(relationship.get("source", ""))
        target = str(relationship.get("target", ""))
        degrees[source] += 1
        degrees[target] += 1
    issues = []
    for node_id, node in node_lookup.items():
        properties = _properties(node)
        if degrees[node_id] != 2 or _is_intersection(properties):
            continue
        issues.append(
            _issue(
                "PSEUDO_NODE",
                "medium",
                "节点度为 2 且未标记为交叉口，可判定为伪节点。",
                [node_id],
                suggestion="合并相邻路段或将节点标记为真实交叉口。",
            )
        )
    return issues
def detect_dead_ends(
    relationships: List[Dict[str, Any]],
) -> List[Dict[str, Any]]:
    degrees: Dict[str, int] = defaultdict(int)
    for relationship in relationships:
        if relationship.get("type") not in TOPOLOGY_RELATIONSHIPS:
            continue
        source = str(relationship.get("source", ""))
        target = str(relationship.get("target", ""))
        degrees[source] += 1
        degrees[target] += 1
    issues = []
    for relationship in relationships:
        if relationship.get("type") not in TOPOLOGY_RELATIONSHIPS:
            continue
        source = str(relationship.get("source", ""))
        target = str(relationship.get("target", ""))
        if degrees[source] > 1 and degrees[target] > 1:
            continue
        issues.append(
            _issue(
                "DEAD_END",
                "high",
                "关系边存在仅一端具有连通度的端点，疑似断头路。",
                [source, target],
                str(relationship.get("id", "")),
                suggestion="检查缺失线段并补齐道路连通关系。",
            )
        )
    return issues
def validate_multilayer_network(
```

```
    nodes: List[Dict[str, Any]],
    relationships: List[Dict[str, Any]],
) -> List[Dict[str, Any]]:
    node_lookup = {_node_id(node): node for node in nodes}
    issues = []
    for relationship in relationships:
        source_id = str(relationship.get("source", ""))
        target_id = str(relationship.get("target", ""))
        source = node_lookup.get(source_id)
        target = node_lookup.get(target_id)
        if source is None or target is None:
            continue
        source_layer = _layer_from(_properties(source))
        target_layer = _layer_from(_properties(target))
        if (
            source_layer is None
            or target_layer is None
            or source_layer == target_layer
        ):
            continue
        relationship_type = str(relationship.get("type", "")).upper()
        if relationship_type in LAYER_CONNECTOR_TYPES:
            continue
        issues.append(
            _issue(
                "CROSS_LAYER_CONNECTION",
                "high",
                "检测到不同高程层之间的直接连接，缺少连接器语义。",
                [source_id, target_id],
                str(relationship.get("id", "")),
                suggestion="补充匝道、互通或跨层连接节点。",
            )
        )
    return issues
def validate_seamless_connections(
    nodes: List[Dict[str, Any]],
    relationships: List[Dict[str, Any]],
    tolerance: float = 1.0,
) -> List[Dict[str, Any]]:
    node_lookup = {_node_id(node): node for node in nodes}
    issues = []
    for relationship in relationships:
        properties = relationship.get("properties", {})
        source_id = str(relationship.get("source", ""))
        target_id = str(relationship.get("target", ""))
        source_node = node_lookup.get(source_id)
        target_node = node_lookup.get(target_id)
        if source_node is None or target_node is None:
            continue
        source_node_coordinate = _coordinate_from(_properties(source_node))
```

```
        target_node_coordinate = _coordinate_from(_properties(target_node))
        edge_source_coordinate = _coordinate_from(
            {
                "x": properties.get("source_x"),
                "y": properties.get("source_y"),
                "lon": properties.get("source_lon"),
                "lat": properties.get("source_lat"),
            }
        )
        edge_target_coordinate = _coordinate_from(
            {
                "x": properties.get("target_x"),
                "y": properties.get("target_y"),
                "lon": properties.get("target_lon"),
                "lat": properties.get("target_lat"),
            }
        )
        endpoint_checks = [
            (source_node_coordinate, edge_source_coordinate),
            (target_node_coordinate, edge_target_coordinate),
        ]
        max_deviation = 0.0
        for node_coordinate, edge_coordinate in endpoint_checks:
            if node_coordinate is None or edge_coordinate is None:
                continue
            max_deviation = max(
                max_deviation,
                hypot(
                    node_coordinate[0] - edge_coordinate[0],
                    node_coordinate[1] - edge_coordinate[1],
                ),
            )
        if max_deviation <= tolerance:
            continue
        issues.append(
            _issue(
                "SEAM_GAP",
                "medium",
                "相邻路段端点坐标偏差超过阈值 {0}。".format(tolerance),
                [source_id, target_id],
                str(relationship.get("id", "")),
                suggestion="吸附端点坐标或拆分错误拓扑连接。",
            )
        )
        issues[-1]["deviation"] = round(max_deviation, 4)
    return issues
def calculate_topology_score(issues: List[Dict[str, Any]]) -> float:
    weights = {
        "PSEUDO_NODE": 0.5,
        "DEAD_END": 3.0,
```

```
        "CROSS_LAYER_CONNECTION": 8.0,
        "SEAM_GAP": 5.0,
    }
    deduction = sum(
        weights.get(str(issue.get("type")), 1.0)
        for issue in issues
    )
    return round(max(0.0, 100.0 - deduction), 2)
def check_topology(
    nodes: List[Dict[str, Any]],
    relationships: List[Dict[str, Any]],
    coordinate_tolerance: float = 1.0,
) -> Dict[str, Any]:
    issues = []
    issues.extend(identify_pseudo_nodes(nodes, relationships))
    issues.extend(detect_dead_ends(relationships))
    issues.extend(validate_multilayer_network(nodes, relationships))
    issues.extend(
        validate_seamless_connections(
            nodes,
            relationships,
            coordinate_tolerance,
        )
    )
    issue_counts: Dict[str, int] = defaultdict(int)
    suggestions = []
    for issue in issues:
        issue_counts[issue["type"]] += 1
        if issue["suggestion"] and issue["suggestion"] not in suggestions:
            suggestions.append(issue["suggestion"])
    return {
        "score": calculate_topology_score(issues),
        "nodeCount": len(nodes),
        "relationshipCount": len(relationships),
        "summary": dict(issue_counts),
        "issues": issues,
        "suggestions": suggestions,
    }
================================================================================================
FILE: backend/learning_path_engine.py
================================================================================================
from collections import defaultdict, deque
from typing import Any, Dict, Iterable, List, Optional, Set, Tuple
from neo4j import READ_ACCESS
DIFFICULTY_ORDER = {
    "入门": 1,
    "基础": 1,
    "简单": 2,
    "中等": 3,
    "进阶": 4,
```

```
    "困难": 5,
    "高级": 5,
    "专家": 6,
}
DIFFICULTY_ALIASES = {
    "beginner": "入门",
    "easy": "基础",
    "medium": "中等",
    "intermediate": "进阶",
    "hard": "困难",
    "advanced": "高级",
}
def difficulty_rank(value: Any) -> int:
    normalized = DIFFICULTY_ALIASES.get(
        str(value or "").strip().lower(),
        str(value or "").strip(),
    )
    return DIFFICULTY_ORDER.get(normalized, 3)
class LearningPathEngine:
    def __init__(
        self,
        nodes: Iterable[Dict[str, Any]],
        relationships: Iterable[Dict[str, Any]],
        max_depth: int = 8,
    ) -> None:
        self.max_depth = max(1, int(max_depth))
        self.nodes: Dict[str, Dict[str, Any]] = {}
        self.prerequisites: Dict[str, List[str]] = defaultdict(list)
        self.successors: Dict[str, List[str]] = defaultdict(list)
        for node in nodes:
            node_id = self._node_id(node)
            if node_id:
                self.nodes[node_id] = node
        for relationship in relationships:
            if relationship.get("type") != "PREREQUISITE_OF":
                continue
            source = str(relationship.get("source", ""))
            target = str(relationship.get("target", ""))
            if not source or not target:
                continue
            self.prerequisites[target].append(source)
            self.successors[source].append(target)
    @staticmethod
    def _node_id(node: Dict[str, Any]) -> str:
        if node.get("id") is not None:
            return str(node["id"])
        properties = node.get("properties", {})
        return str(properties.get("kp_id", ""))
    @staticmethod
    def _properties(node: Dict[str, Any]) -> Dict[str, Any]:
```

```
    properties = node.get("properties", node)
    return properties if isinstance(properties, dict) else {}
def node(self, knowledge_point_id: str) -> Optional[Dict[str, Any]]:
    return self.nodes.get(str(knowledge_point_id))
def prerequisite_chain(
    self,
    knowledge_point_id: str,
    max_depth: Optional[int] = None,
) -> List[Dict[str, Any]]:
    root = str(knowledge_point_id)
    depth_limit = min(
        self.max_depth,
        int(max_depth) if max_depth is not None else self.max_depth,
    )
    visited: Set[str] = set()
    queue = deque((predecessor, 1) for predecessor in self.prerequisites[root])
    collected: List[Tuple[int, str]] = []
    while queue:
        node_id, depth = queue.popleft()
        if node_id in visited or depth > depth_limit:
            continue
        visited.add(node_id)
        collected.append((depth, node_id))
        for predecessor in self.prerequisites[node_id]:
            queue.append((predecessor, depth + 1))
    collected.sort(
        key=lambda item: (
            -item[0],
            difficulty_rank(self._properties(self.nodes[item[1]]).get("difficulty"))
            if item[1] in self.nodes
            else 3,
        )
    )
    return [
        self._path_item(node_id, "prerequisite", depth)
        for depth, node_id in collected
    ]
def build_learning_path(
    self,
    target_knowledge_point_id: str,
    include_successors: int = 2,
) -> List[Dict[str, Any]]:
    target_id = str(target_knowledge_point_id)
    path = self.prerequisite_chain(target_id, self.max_depth)
    if target_id in self.nodes:
        path.append(self._path_item(target_id, "target", 0))
    successors = sorted(
        self.successors[target_id],
        key=lambda node_id: (
            difficulty_rank(
```

```
                self._properties(self.nodes[node_id]).get("difficulty")
            )
            if node_id in self.nodes
            else 3,
            self._properties(self.nodes[node_id]).get("sort_order", 9999)
            if node_id in self.nodes
            else 9999,
        ),
    )
    for node_id in successors[: max(0, int(include_successors))]:
        path.append(self._path_item(node_id, "successor", 1))
    return path
def recommend(
    self,
    knowledge_point_id: str,
) -> Dict[str, Any]:
    target_id = str(knowledge_point_id)
    prerequisite_items = self.prerequisite_chain(target_id)
    successor_items = [
        self._path_item(node_id, "successor", 1)
        for node_id in self.successors[target_id]
    ]
    successor_items.sort(
        key=lambda item: (
            item["difficultyRank"],
            item["sortOrder"],
        )
    )
    return {
        "knowledgePoint": self._path_item(target_id, "current", 0),
        "prerequisites": prerequisite_items,
        "successors": successor_items,
        "recommendedPath": self.build_learning_path(target_id),
    }
def filter_by_difficulty(
    self,
    difficulty: str,
    module_id: Optional[str] = None,
) -> List[Dict[str, Any]]:
    expected_rank = difficulty_rank(difficulty)
    result = []
    for node_id, node in self.nodes.items():
        properties = self._properties(node)
        if module_id and str(properties.get("module_id", "")) != str(module_id):
            continue
        if difficulty_rank(properties.get("difficulty")) != expected_rank:
            continue
        result.append(self._path_item(node_id, "knowledge", 0))
    result.sort(key=lambda item: (item["sortOrder"], item["name"]))
    return result
```

```
    def _path_item(
        self,
        node_id: str,
        role: str,
        depth: int,
    ) -> Dict[str, Any]:
        node = self.nodes.get(str(node_id), {})
        properties = self._properties(node)
        return {
            "id": str(node_id),
            "name": properties.get("name", "未命名知识点"),
            "description": properties.get("description"),
            "difficulty": properties.get("difficulty", "未标注"),
            "difficultyRank": difficulty_rank(properties.get("difficulty")),
            "importance": properties.get("importance"),
            "moduleId": properties.get("module_id"),
            "sortOrder": properties.get("sort_order", 9999),
            "role": role,
            "depth": int(depth),
        }
def load_learning_graph(
    driver,
    database: str,
    max_depth: int = 8,
) -> LearningPathEngine:
    with driver.session(
        database=database,
        default_access_mode=READ_ACCESS,
    ) as session:
        node_records = list(
            session.run(
                """
                MATCH (knowledge:KnowledgePoint)
                OPTIONAL MATCH (knowledge)-[:BELONGS_TO]->(module:Module)
                RETURN knowledge, module.module_id AS module_id
                """,
                timeout=10.0,
            )
        )
        relationship_records = list(
            session.run(
                """
                MATCH (source:KnowledgePoint)-[relationship:PREREQUISITE_OF]->
                      (target:KnowledgePoint)
                RETURN source.kp_id AS source, target.kp_id AS target
                """,
                timeout=10.0,
            )
        )
    nodes = []
```